

گزارش روند اجرای پروژه طبقه‌بندی وسایل نقلیه ترافیکی

مستندسازی تصمیم‌ها، آماده‌سازی داده و آزمایش‌های Ablation

ساختار گزارش مطابق ساختار پوشه‌های پروژه در VS Code

مسعود ذوقیان

لینک پروژه : https://github.com/masoud-zoghian/Traffic_Vehicle_Classification

پاییز 1405

بوت کمپ هوش مصنوعی مکتب شریف

1) خلاصه اجرایی پروژه

هدف این پروژه ساخت یک سیستم طبقه‌بندی تصویر برای شناسایی وسایل نقلیه ترافیکی با استفاده از شبکه‌های عصبی کانولوشنی (CNN) و در ادامه روش‌های یادگیری انتقالی است. تمرکز اصلی روی آماده‌سازی صحیح داده، جلوگیری از نشت داده (Data Leakage)، ساخت یک مدل پایه و بررسی اثر چند تغییر معماری/آموزشی به‌صورت آزمایش‌های مستقل بوده است.

یک تصمیم مهم در کل پروژه این بود که هر تغییر مهم در یک پوشه و Notebook جداگانه ثبت شود. بنابراین نتیجه هر آزمایش قابل مقایسه و قابل بازتولید است و مشخص می‌شود که چه چیزی نسبت به مدل پایه تغییر کرده است.

موضوع	تصمیم نهایی
تعداد کلاس‌ها	ambulance, autobus, kamyun, minibus, savari, taxi, vanet
تعداد تصاویر یکتا	3868 تصویر
Train / Validation	3094 / 774 تصویر
روش Split	Stratified Split با $random_state=42$
Test Set	جدا و دست‌نخورده نگه داشته شد
اندازه ورودی CNN	$224 \times 224 \times 3$
Normalization	Std=[0.229,0.224,0.225] و Mean=[0.485,0.456,0.406]
Batch Size پایه	32
Loss پایه	Cross-Entropy
Optimizer پایه	Adam با $learning\ rate=0.001$
مدل پایه	CNN دو بلوکی با Conv + ReLU + Pooling

2) انتخاب و آماده‌سازی داده‌ها

در شروع پروژه چند منبع داده وجود داشت. به‌جای انتخاب تنها یک پوشه، منابع آموزشی موجود با هم بررسی و برای ساخت یک مجموعه آموزش/اعتبارسنجی واحد استفاده شدند.

2.1 - منابع داده مورد استفاده

- DATA/dataset/train
- DATA/dataset/clean
- DATA/datasetv2_TrainClean/train
- DATA/datasetv2_TrainClean/clean

2.2 - تصمیم‌های مربوط به برچسب‌ها

- نسخه نهایی داده دارای 7 کلاس است: ambulance, autobus, kamyun, minibus, savari, taxi, vanet.
- کلاس neysan به‌صورت دستی در کلاس vanet ادغام شد.
- کلاس kamyunet به‌صورت دستی در کلاس kamyun ادغام شد.
- داده‌هایی که قبلاً در مجموعه unclean قرار داشتند، پیش از این مرحله به clean تبدیل شده بودند؛ بنابراین در این نسخه مجموعه مستقل unclean وجود نداشت.
- پس از این تصمیم‌ها، تمام تصاویر با 7 برچسب نهایی وارد فرایند ساخت Manifest شدند.

2.3 - بررسی Duplicate و جلوگیری از تکرار داده

برای جلوگیری از این که یک تصویر یکسان در چند منبع وجود داشته باشد و بعداً به شکل غیر مستقیم وارد Train و Validation شود، تصاویر با MD5 بر اساس بایت خام فایل بررسی شدند.

- 16 گروه Duplicate شناسایی شد.
- تمام Duplicate ها بین train اولیه و clean اولیه بودند.
- Duplicate متناقض بین دو کلاس مشاهده نشد؛ یعنی یک تصویر یکسان با دو برچسب متفاوت پیدا نشد.
- هیچ Duplicate ای مربوط به مجموعه v2 نبود.
- پس از حذف تکراری‌ها، تعداد تصاویر یکتا به 3868 رسید.

2.4 - تصمیم درباره Test Set

مجموعه Test برای ارزیابی نهایی کنار گذاشته شد و در مرحله آماده‌سازی Train/Validation وارد Split نشد. این تصمیم برای جلوگیری از Data Leakage و حفظ اعتبار ارزیابی نهایی اتخاذ شد.

3 ساخت Manifest و تقسیم داده

به جای این که مسیر تصاویر در هر Notebook به صورت دستی جمع‌آوری شود، یک Manifest مرکزی ساخته شد. این Manifest مسیر تصویر و برچسب آن را ثبت می‌کند و سپس Train و Validation از روی همان اطلاعات ساخته می‌شوند.

- data_splits/all_images.csv: فهرست کامل تصاویر یکتا و برچسب‌ها.
- data_splits/train.csv: زیرمجموعه آموزش.
- data_splits/val.csv: زیرمجموعه اعتبارسنجی.
- Split به صورت Stratified انجام شد تا نسبت کلاس‌ها در Train و Validation تا حد ممکن حفظ شود.
- random_state=42 برای ثابت ماندن Split انتخاب شد.
- هیچ Overlap بین Train و Validation وجود نداشت.

3.1 - آمار نهایی داده

کلاس	Train	Validation	Total
ambulance	289	72	361
autobus	370	93	463
kamyun	756	189	945
minibus	314	78	392
savari	398	100	498
taxi	387	97	484
vanet	580	145	725
جمع	3094	774	3868

4) منطق کلی ساختار پوشه‌های پروژه

برای اینکه آزمایش‌ها با یکدیگر قاطی نشوند، هر مرحله در پوشه مستقل قرار گرفت. پوشه‌های آزمایشگاهی از 06_training به بعد هر کدام فقط یک عامل اصلی را تغییر می‌دهند و سایر اجزای مدل تا حد ممکن ثابت نگه داشته می‌شوند.

پوشه	هدف اصلی	خروجی مهم
04_dataset_transforms	تعریف Dataset و تبدیل تصاویر	dataset.ipynb
05_dataloader	ساخت DataLoader و Batch ها	Train/Val DataLoader
06_training	مدل پایه و آموزش اولیه	best_baseline_cnn.pth
07_augmentation	بررسی Data Augmentation	best_augmentation_model.pth
08_dropout	بررسی Dropout	best_dropout_model.pth
09_pooling	مقایسه AvgPool و MaxPool	best_pooling_model.pt
10_weight_decay	بررسی Regularization با Weight Decay	best_weight_decay_model.pt
11_scheduler	بررسی کاهش Learning Rate	best_scheduler_model.pt
12_balanced_batches	بررسی Balanced Sampling	best_balanced_batches_model.pt
13_loss_comparison	مقایسه BCE و Cross-Entropy	best_ce_model.pt / best_bce_model.pt

5) پوشه 04_dataset_transforms

هدف این پوشه ساخت یک Dataset استاندارد برای پروژه بود تا تمام Notebook های بعدی از یک روش ثابت برای خواندن تصویر، تبدیل تصویر به Tensor و تبدیل نام کلاس به عدد استفاده کنند.

- کلاس TrafficVehicleDataset از torch.utils.data.Dataset ساخته شد.
- در __getitem__ مسیر تصویر از DataFrame خوانده می‌شود.
- تصویر با PIL باز و با convert('RGB') به سه کانال استاندارد تبدیل می‌شود.
- نام کلاس با class_to_idx به عدد تبدیل می‌شود.
- Transform در زمان خواندن تصویر روی آن اعمال می‌شود.
- کلاس‌ها با یک ترتیب ثابت به اندیس 0 تا 6 نگاشت شدند تا همه آزمایش‌ها برچسب‌گذاری یکسان داشته باشند.

Transform پایه:

- Resize به 224×224
 - ToTensor
 - Normalize با Mean و Std متداول ImageNet
- در Transform پایه هیچ Random Augmentation قرار داده نشد تا مدل پایه نقش مرجع داشته باشد و بعداً اثر Augmentation به صورت جداگانه قابل اندازه‌گیری باشد.

6) پوشه 05_dataloader

هدف این پوشه آماده کردن داده‌های Train و Validation برای ورود به مدل در قالب Batch های ثابت بود. این مرحله عملاً رابط بین Dataset و Training Loop است.

- Train DataLoader با batch_size=32 ساخته شد.

- shuffle=True برای Train استفاده شد تا ترتیب نمونه‌ها در هر Epoch تصادفی شود.
- Validation DataLoader با shuffle=False ساخته شد تا ارزیابی پایدار و قابل تکرار باشد.
- هر Batch شامل image tensor و label tensor است.
- ساخت DataLoader به صورت مستقل باعث شد تمام آزمایش‌ها از یک pipeline ورودی مشترک استفاده کنند.

7 پوشه — 06_training مدل پایه

این پوشه نقطه شروع مقایسه تمام آزمایش‌ها بود. ابتدا یک CNN ساده و قابل فهم ساخته شد تا قبل از استفاده از مدل‌های پیچیده‌تر، یک Baseline واقعی برای پروژه داشته باشیم.

7.1 - معماری مدل پایه

- Conv2d: ورودی 3 کاناله به 32 کانال
- ReLU
- MaxPool2d با ضریب 2
- Conv2d: 32 به 64 کانال
- ReLU
- MaxPool2d با ضریب 2
- Flatten
- Linear از 64×56×56 به 7 کلاس

با ورودی 224×224، دو مرحله Pooling اندازه فضایی را از 224 به 112 و سپس به 56 کاهش می‌دهند. بنابراین قبل از لایه Linear تعداد ویژگی‌ها برابر 64×56×56 یعنی 200704 است.

7.2 - تنظیمات آموزش مدل پایه

پارامتر	مقدار
Loss	CrossEntropyLoss
Optimizer	Adam
Learning Rate	0.001
Weight Decay	0
Batch Size	32
Input Size	224×224
تعداد کلاس	7

بهترین نتیجه مدل پایه در 8 Epoch با Validation Accuracy حدود 79.98 % به دست آمد. این مدل به عنوان مرجع اصلی برای مقایسه آزمایش‌های بعدی ذخیره شد.

8 آزمایش‌های Ablation

منطق Ablation این بود که هر بار فقط یک مؤلفه اصلی تغییر کند و نتیجه آن با Baseline مقایسه شود. این روش کمک می‌کند بفهمیم بهبود یا افت عملکرد دقیقاً به کدام تصمیم مربوط است.

8.1 - پوشه augmentation_07

هدف بررسی این بود که آیا ایجاد augmentation در تصاویر Train باعث افزایش Generalization می‌شود یا خیر؟

- RandomHorizontalFlip با احتمال 0.5
 - RandomRotation تا 10 درجه
 - سپس ToTensor و Normalize
 - معماری مدل، Loss و Optimizer مانند Baseline نگه داشته شدند.
- نتیجه: بهترین Validation Accuracy حدود 68.99 % بود؛ بنابراین در این آزمایش Augmentation نسبت به Baseline باعث افت عملکرد شد.
- نکته تحلیلی: این نتیجه به معنی بد بودن Augmentation به صورت کلی نیست؛ فقط نشان می‌دهد این دو تبدیل و این شدت در این دیتاست و این مدل، برای این آزمایش مناسب‌تر از Baseline نبوده‌اند.

8.2 - پوشه dropout_08

هدف بررسی اثر Regularization با Dropout بود Dropout. با احتمال 0.5 قبل از لایه Linear قرار گرفت.

- Dropout(p=0.5) فقط در بخش Classifier اضافه شد.
 - بقیه اجزای مدل مانند Baseline باقی ماند.
- نتیجه: بهترین Validation Accuracy حدود 83.72% بود؛ یعنی بالاتر از 79.98 % Baseline.

8.3 - پوشه pooling_09

در این آزمایش تنها نوع Pooling تغییر میکند تا بررسی شود آیا Average Pooling نسبت به Max Pooling برای ویژگی‌های این تصاویر مناسب‌تر است؟

- AvgPool2d با MaxPool2d جایگزین شد.
 - ساختار سایر بخش‌ها ثابت نگه داشته شد.
- نتیجه: بهترین Validation Accuracy حدود 85.27% بود؛ این آزمایش از Baseline بهتر عمل کرد و یکی از بهترین نتایج Ablation تا این مرحله بود.

8.4 - پوشه weight_decay_10

هدف بررسی Regularization از طریق محدود کردن اندازه وزن‌ها بود، بدون تغییر معماری شبکه.

- Adam با weight_decay=0.0001
 - Learning Rate برابر 0.001 باقی ماند.
 - Loss همان Cross-Entropy باقی ماند.
- نتیجه: بهترین Validation Accuracy حدود 85.14% بود؛ بهتر از Baseline و بسیار نزدیک به Avg Pooling.

8.5 - پوشه scheduler_11

هدف این آزمایش بررسی این بود که آیا کاهش مرحله‌ای Learning Rate باعث می‌شود مدل در مراحل پایانی آموزش بهتر همگرا شود؟

- StepLR استفاده شد.

- step_size=3

- gamma=0.1

- Learning Rate اولیه = 0.001

- در نتیجه LR بعد از هر 3 Epoch یک‌دهم شد.

نتیجه : بهترین Validation Accuracy حدود 82.17% بود؛ بنابراین این تنظیم Scheduler در این آزمایش از Baseline ضعیف‌تر بود.

8.6 - پوشه 12_balanced_batches

داده‌ها کاملاً متوازن نبودند و تعداد نمونه‌های کلاس‌ها متفاوت بود. در این آزمایش به‌جای تغییر خود تصاویر، نحوه نمونه برداری از Train تغییر داده شد تا کلاس‌های کم نمونه فرصت بیشتری برای حضور در Batch ها داشته باشند.

- WeightedRandomSampler استفاده شد.

- وزن نمونه‌ها بر اساس معکوس تعداد نمونه‌های کلاس مربوطه تعیین شد.

- هدف، نزدیک‌شدن توزیع نمونه‌های دیده شده توسط مدل به حالت متوازن بود.

نتیجه Accuracy: حدود 82.68% ، Macro Precision حدود 84.15% ، Macro Recall حدود 82.47% و Macro F1 حدود 82.81% گزارش شد. بنابراین Balanced Sampling در این تنظیمات از Baseline بهتر شد.

نکته مهم: با توجه به نامتوازن بودن داده‌ها، فقط Accuracy معیار کافی نیست؛ به همین دلیل در گزارش آزمایش‌ها Precision، Recall و F1 به‌صورت Macro نیز بررسی شدند.

9 پوشه 13_loss_comparison

هدف این پوشه مقایسه دو روش رایج برای تعریف Loss در مسئله چندکلاسه بود Cross-Entropy و Binary Cross-Entropy. از آنجا که پروژه 7 کلاس دارد و هر تصویر فقط یک کلاس صحیح دارد، Cross-Entropy انتخاب طبیعی‌تر برای مدل چندکلاسه است؛ با این حال BCE به‌عنوان یک آزمایش مقایسه‌ای اجرا شد.

9.1 - آزمایش Cross-Entropy

- Loss: nn.CrossEntropyLoss()

- خروجی مدل به‌صورت logits برای 7 کلاس باقی ماند.

- برای Prediction از argmax روی logits استفاده شد.

- در یکی از اجراهای ثبت شده، Best Validation Accuracy برابر 81.91% در Epoch 10 بود.

9.2 - آزمایش BCE

برای BCE خروجی مدل با همان تعداد کلاس‌ها استفاده شد و هنگام ارزیابی sigmoid روی خروجی اعمال شد؛ سپس کلاس با بیشترین مقدار احتمال انتخاب شد.

نتیجه نهایی ثبت‌شده: BCE بهترین Epoch برابر 8 و Validation Accuracy برابر 85.40% بود.

معیار	BCE
Accuracy	0.8540
Macro Precision	0.8592
Macro Recall	0.8507

Macro F1	0.8538
----------	--------

9.3 - نتیجه ارزیابی BCE بر اساس کلاس‌ها

کلاس	Precision	Recall	F1	Support
ambulance	0.77	0.76	0.77	72
autobus	0.94	0.78	0.85	93
kamyun	0.79	0.84	0.81	189
minibus	0.83	0.86	0.84	78
savari	0.86	0.89	0.87	100
taxi	0.94	0.94	0.94	97
vanet	0.89	0.88	0.89	145

ماتریس خطای BCE نشان داد بیشترین خطاها عمدتاً بین کلاس‌های ظاهراً مشابه، به‌خصوص kamyun ، minibus و vanet، رخ می‌دهند.

10) جمع‌بندی نتایج آزمایش‌ها

آزمایش	پوشه	Best Val Accuracy	تفسیر کوتاه
Baseline CNN	06_training	79.98%	مدل مرجع
Augmentation	07_augmentation	68.99	کاهش نسبت به Baseline
Dropout	08_dropout	83.72%	کاهش نسبت به Baseline
Average Pooling	09_pooling	85.27%	بهبود نسبت به Baseline
Weight Decay	10_weight_decay	85.14%	بهبود نسبت به Baseline
Scheduler	11_scheduler	82.17%	کاهش نسبت به Baseline
Balanced Batches	12_balanced_batches	82.68%	بهبود محسوسی ایجاد نکرد
Cross-Entropy	13_loss_comparison	81.91%	نتیجه ثبت شده
BCE	13_loss_comparison	85.40%	نتیجه نهایی ثبت شده

یک نکته عملی نیز ثبت شد: فقط تعیین Seed کافی نیست؛ مدل باید بعد از تعیین Seed از ابتدا ساخته شود. اگر Notebook فقط بخش Training را دوباره اجرا کند، آموزش از وزن‌های قبلی ادامه پیدا می‌کند و نتیجه دیگر اجرای کاملاً مستقل محسوب نمی‌شود.

11) روش ذخیره بهترین مدل و ارزیابی

در Training Loop ، عملکرد Validation در هر Epoch بررسی شد و بهترین وزن مدل بر اساس Validation Accuracy ذخیره شد. به این ترتیب برای ارزیابی نهایی از بهترین Epoch استفاده شد، نه الزاماً آخرین Epoch .

- برای ارزیابی، مدل ذخیره‌شده دوباره با load_state_dict بارگذاری شد.
- model.eval() برای خاموش شدن رفتارهای مخصوص آموزش مانند Dropout استفاده شد.
- torch.no_grad() برای جلوگیری از محاسبه گرادیان در Evaluation استفاده شد.

12) تصمیم‌ها و درس‌های مهم تا این مرحله

- داده قبل از مدل‌سازی باید Audit شود؛ صرفاً داشتن تعداد زیادی تصویر به معنی داشتن دیتاست سالم نیست.
- Duplicate Detection قبل از Split اهمیت دارد، چون Duplicate می‌تواند باعث خوش‌بینی کاذب در Validation شود.
- استفاده از Manifest مرکزی باعث می‌شود همه آزمایش‌ها دقیقاً روی همان Train/Validation اجرا شوند.
- Baseline قبل از هر Ablation لازم است تا اثر هر تغییر قابل اندازه‌گیری باشد.
- در داده نامتوازن، Macro Precision/Recall/F1 برای تحلیل عملکرد کلاس‌ها مفیدتر از تکیه صرف بر Accuracy هستند.
- هر آزمایش بهتر است فقط یک عامل اصلی را تغییر دهد تا علت تغییر عملکرد قابل تشخیص باشد.
- نتایج یک اجرای تصادفی لزوماً با اجرای بعدی یکسان نیست؛ Seed و تنظیمات Reproducibility باید ثبت شوند.

13) نمای کلی ساختار پوشه‌ها در این مرحله

پوشه	محتوا / هدف
04_dataset_transforms	Resize/Normalize ، class mapping، Dataset و خواندن تصاویر
05_dataloader	Batch سازی، Shuffle برای Train و DataLoader برای Validation
06_training	Adam ، Cross-Entropy، Baseline CNN و ذخیره بهترین مدل
07_augmentation	RandomRotation و RandomHorizontalFlip
08_dropout	Dropout با $p=0.5$
09_pooling	جایگزینی AvgPool با MaxPool
10_weight_decay	Adam با $\text{weight_decay}=0.0001$
11_scheduler	StepLR با $\text{step_size}=3$ و $\text{gamma}=0.1$
12_balanced_batches	WeightedRandomSampler برای نمونه‌برداری متوازن‌تر
13_loss_comparison	مقایسه Cross-Entropy و BCE